LEPL1503/LSINC1503 - Cours 5

O. Bonaventure, B. Legat, M. Baerts

☐ Full Width Mode ☐ Present Mode

Table of Contents

Clarifications projet

- Il est important de lire la partie multithread du syllabus pour savoir faire la deuxième partie
- Amenez le Raspberry aux séances de TP pour que votre tuteur puissent vérifier que vous savez l'utiliser et que votre code fonctionne dessus. Les mesures devront être faites sur le Raspeberry pour être valable. On ne considère pas les mesures sur une autre plateforme.
- N'oubliez de régulièrement récupérer les modifications réalisées sur le dépôt parent.
- Rappel : l'utilisation de logiciels d'IA générative tels que chatGPT, GitHub copilot, ... est interdite pour l'écriture de code source en langage C et les reviews. Par ailleurs, les sources d'information externes doivent être systématiquement citées en respectant les normes de référencement bibliographique.

Chacun doit contribuer sur le Git

- Pour valider votre contribution au projet, il est nécessaire que vous ayez des merge requests en votre nom qui sont mergées avec des commits également en votre nom
- Ces merge requests doivent avoir été finies (donc passer les tests) puis mergées
- Elles doivent apporter une contribution significatives au code, par exemple pas juste modifier le README ou reformatter
- L'excuse du peer coding (vous programmez à deux sur un ordinateur) n'est pas valide
- Ne pas savoir exécuter le code chez soi n'est pas une excuse. Git est facile à installer sous n'importe quelle platforme et les tests s'exécute sur le runner GitLab.

Vérifiez qu'il n'y a pas de fuites de mémoires

Lors de la correction, nous vérifierons qu'il n'y a pas de fuites de mémoire avec Valgrind

```c
#include <stdlib.h>

void f(void)
{
    int *x = malloc(10 * sizeof(int));
    x[10] = 0; // problem 1: heap block overrun
} // problem 2: memory leak -- x not freed

int main(void)
{
    f();
    return 0;
} // From https://valgrind.org/docs/manual/quick-start.html
```

```julia
compile_and_run(valgrind, valgrind = true, verbose = true, cflags = valgrind_debug ? ["-g"] : String[])
```

```
Compiling : `/home/runner/.julia/artifacts/207eb5b8330e24674fe59b50d72f4b3d946219c8/tools/clang /tmp/jl_SFhMy5/main.c -o /tmp/jl_SFhMy5/bin`
```

```
Running : `valgrind /tmp/jl_SFhMy5/bin`
```

```
==10502== Memcheck, a memory error detector
==10502== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==10502== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==10502== Command: /tmp/jl_SFhMy5/bin
==10502==
==10502== Invalid write of size 4
==10502==    at 0x10915A: f (in /tmp/jl_SFhMy5/bin)
==10502==    by 0x109183: main (in /tmp/jl_SFhMy5/bin)
==10502==  Address 0x4a79068 is 0 bytes after a block of size 40 alloc'd
==10502==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==10502==    by 0x109151: f (in /tmp/jl_SFhMy5/bin)
==10502==    by 0x109183: main (in /tmp/jl_SFhMy5/bin)
==10502==
==10502==
==10502== HEAP SUMMARY:
==10502==     in use at exit: 40 bytes in 1 blocks
==10502==   total heap usage: 1 allocs, 0 frees, 40 bytes allocated
==10502==
==10502== LEAK SUMMARY:
==10502==    definitely lost: 40 bytes in 1 blocks
==10502==    indirectly lost: 0 bytes in 0 blocks
==10502==      possibly lost: 0 bytes in 0 blocks
```

▶ **Pourquoi `valgrind` ne donne-t-il pas le numéro de ligne ?**

Interface par fichiers

On ne partage normalement pas de fichiers binaires (ex : `.o`) par `git` car ce n'est pas portable (ex : ce `.o` ne marche pas sur Mac OS si compilé sur un Raspberry). Lorsque des fichiers auto-générés apparaissent dans la sortie de `git status`, ajoutez-les dans le `.gitignore`. Supprimez-les de git avec `git rm` s'ils ont été ajouté précédemment par accident, ce qui ne devrait pas arriver si `git add -p` est toujours utilisé au lieu de `git add .`.

C'est une bonne chose d'écrire des unit tests, mais comme les signatures des fonctions sont libres, nous utiliserons l'interface uniformisée en ligne de commande pour testez votre code. Assurez-vous qu'elle fonctionne comme demandée.

Le Makefile génère un exécutable. Vous pouvez l'utiliser de la façon suivante :

```
./<nom du fichier> [arguments]
```

Debugging

- Ne pas oublier d'utiliser `-g` lors de la compilation
- `lldb` : développé par LLVM comme `clang` mais fonctionne aussi avec des binaires compilés avec `gcc`, plus facile à installer sur Mac OS
- `gdb` : développé par GNU comme `gcc` mais fonctionne aussi avec des binaires compilés avec `clang`
- Il existe des interfaces graphiques pour ces débogueur, ex : Seer (`apt install seergdb`), VSCode, lldbg, gdbgui, etc.
- Utiliser un débogueur, c'est un bon réflex !
- Utilisez `-fsanitize=address` à la compilation pour que les bornes soient vérifiées.
    - Le code est peut-être un peu moins rapide donc c'est à désactiver pour les mesures
    - mais il n'y a pas de bonne raison de ne pas l'utiliser dans la phase de développement!

Exemple sans sanitize

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i = 0;
    int a[2];
    a[-1] = 1000000;
    a[2] = -1000000;
    printf("&i = %p\n", &i);
    printf("&a = %p\n", &a);
    printf("i = %d\n", i);
    return EXIT_SUCCESS;
}
```

```
Compiling : `/home/runner/.julia/artifacts/207eb5b8330e24674fe59b50d72f4b3d946219c8/tools/clang -Wno-array-bounds -g /tmp/jl_Dh3eg8/main.c -o /tmp/jl_Dh3eg8/bin`
```

```
Running : `/tmp/jl_Dh3eg8/bin`
```

```
&i = 0x7ffc49e54a68
&a = 0x7ffc49e54a60
i = -1000000
```

Exemple avec sanitize

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i = 0;
    int a[2];
    a[-1] = 1000000;
    a[2] = -1000000;
    printf("&i = %p\n", &i);
    printf("&a = %p\n", &a);
    printf("i = %d\n", i);
    return EXIT_SUCCESS;
}
```

```
Compiling : `/home/runner/.julia/artifacts/207eb5b8330e24674fe59b50d72f4b3d946219c8/tools/clang -Wno-array-bounds -fsanitize=address -g /tmp/jl_AIyd97/main.c -o /tmp/jl_AIyd97/bin`
```

```
Running : `/tmp/jl_AIyd97/bin`
```

```
ProcessFailedException
```

```
=================================================================
==10510==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7f16bc60002c at pc 0x5601c4195bca bp 0x7ffd1522f9f0 sp 0x7ffd1522f9e8
WRITE of size 4 at 0x7f16bc60002c thread T0
    #0 0x5601c4195bc9  (/tmp/jl_AIyd97/bin+0x162bc9)
    #1 0x7f16be62a1c9  (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId: 8e9fd827446c24067541ac5390e6f527fb5947bb)
    #2 0x7f16be62a28a  (/lib/x86_64-linux-gnu/libc.so.6+0x2a28a) (BuildId: 8e9fd827446c24067541ac5390e6f527fb5947bb)
    #3 0x5601c405e3f4  (/tmp/jl_AIyd97/bin+0x2b3f4)

Address 0x7f16bc60002c is located in stack of thread T0 at offset 44 in frame
    #0 0x5601c4195aaf  (/tmp/jl_AIyd97/bin+0x162aaf)

  This frame has 2 object(s):
    [32, 36) 'i' (line 6)
    [48, 56) 'a' (line 7) <== Memory access at offset 44 underflows this variable
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext or vfork
```

The End

```julia
import Pkg
```

```julia
Pkg.activate(".")
```

```
Activating project at `~/work/LEPL1503/LEPL1503/Lectures`
```